

Python 面试学习计划 与重点题库

基础语法 · 高频算法 · AI / 深度学习岗位专项 分阶段学习路线 · 分层题库 ·
最优解与解析

适用：Python 初学者、实习 / 校招求职者、AI 算法岗位准备者

2026 年学习版

目录

目录随文档生成自动更新；电子版可点击条目跳转。

目录	2
1. 学习目标与使用方法	4
使用流程	4
面试答题框架	4
2. 按重要性排序的学习路线	5
必须掌握 (★★★★★)	5
高频强化 (★★★★☆)	5
专项与进阶	5
3. 分阶段与按周学习计划	6
10 周执行表	6
每日 / 每周执行方式	6
4. Python 基础、容器与复杂度清单	7
函数与面向对象	7
list / dict / set / deque / heapq	7
复杂度直觉	7
5. 算法思想速通：信号、模板、易错点	8
5.1 哈希表	8
5.2 双指针	8
5.3 二分查找	8
5.4 滑动窗口	8
5.5 前缀和	8
5.6 排序与贪心	8
5.7 栈 / 单调栈	8
5.8 队列与 BFS	8
5.9 DFS、树与图	8
5.10 递归与回溯	9
5.11 动态规划	9
5.12 并查集	9
5.13 堆	9
6. AI / 深度学习算法岗位专项	10
NumPy 与张量思维	10
PyTorch 训练流程	10
医学图像分割	10
7. Python 面试重点题库	11
7.1 选择题：简单	11
7.2 选择题：中等	11

- 7.3 选择题：困难 11
- 7.4 简答题：简单 12
- 7.5 简答题：中等 12
- 7.6 简答题：困难 12
- 7.7 编程题：简单 13
- 7.8 编程题：中等 15
- 7.9 编程题：困难 18
- 8. 复习与模拟面试安排 21
 - 错题复盘卡 21
 - 四次模拟面试 21
 - 评分量表（每道编码题 10 分） 21
 - 最终检查清单 21

1. 学习目标与使用方法

面试难度并不由 Python 语法是否复杂决定，而取决于能否从题目条件中识别出合适的算法策略、稳定处理边界、并说明复杂度。两个升序列表合并只使用 while、if、append，却考察你是否利用“有序”条件把重新排序的 $O((m+n)^2)$ 优化为 $O(m+n)$ 。

完成标准

简单题能在 15-25 分钟独立完成，中等题能在 35 分钟内写出主逻辑；每题都能说明时间 / 空间复杂度，并用普通、空、重复、极值至少 4 类用例自测。

使用流程

- 先读题并口述约束：输入是否为空、是否允许重复、是否已排序、是否允许修改原数据。
- 先给可行的直接方案，再利用题目条件优化；写主循环前先定义循环不变量。
- 遮住答案作答；卡住超过限时先看“题型提示”，仍无思路才看最优解。
- 错题按第 1、3、7、14 天重写，重写时不能看自己原来的代码。
- 每周做一次限时模拟，并练习用两分钟解释一道题：思路、正确性、复杂度、边界。

面试答题框架

步骤	应该做什么	注意
澄清	确认输入、输出、重复、空值、排序与修改权限。	把隐藏约束说出来。
朴素解	给出最直接的工作方案及复杂度。	展示拆解能力，避免沉默。
利用条件	有序、连续、唯一、图、单调性分别能带来什么优化？	选择 hash / 双指针 / 二分 / 窗口等。
编码	先写主循环与状态，再写边界与剩余元素。	变量名与更新顺序要一致。
验证	走普通、重复、空、极值用例。	最后汇报时间和空间复杂度。

2. 按重要性排序的学习路线

必须掌握 (★★★★★)

主题	识别信号	核心题与能力
哈希表 dict / set	查找、去重、计数、补数、分组。	Two Sum、频率统计、字母异位词；平均 $O(1)$ 查找。
双指针	有序数组、两端夹逼、原地修改。	有序合并、移零、去重、三数之和。
二分查找	已排序或“可行性”具有单调性。	查值、左右边界、答案二分；每轮砍掉一半。
滑动窗口	连续子串 / 子数组的最长、最短、K 约束。	最长无重复子串、最小覆盖子串、固定窗口。
复杂度分析	所有算法题。	理解 $O(1)$ 、 $O(\log n)$ 、 $O(n)$ 、 $O(n \log n)$ 、 $O(n^2)$ 。

高频强化 (★★★★☆)

主题	何时使用	典型题
前缀和 / 差分	区间和、多次查询、连续和等于目标。	区间和、和为 K 的子数组。
排序与贪心	排序后局部选择可得到最优或可行解。	区间合并、会议室、跳跃游戏。
栈 / 单调栈	嵌套匹配、最近未解决元素、下一个更大值。	有效括号、每日温度、柱状图。
队列 / deque	按层访问、无权最短步数、扩散。	层序遍历、迷宫 BFS。
树与图 DFS / BFS	连通性、路径、层数。	岛屿数量、克隆图、课程表。
递归 / 回溯	枚举所有组合、排列、放置方案。	子集、全排列、N 皇后。
动态规划	最优子结构、重叠子问题。	爬楼梯、打家劫舍、背包、LIS。

专项与进阶

主题	识别	要点
并查集	动态连通、集合合并、检测无向图环。	find + union；路径压缩、按秩合并。
堆 heapq	Top K、反复取最小 / 最大、多路合并。	Python 是最小堆，push / pop 是 $O(\log n)$ 。
分治 / 位运算	可递归拆解、状态压缩、二进制集合。	归并、快速选择、掩码状态。

3. 分阶段与按周学习计划

阶段	建议时长	内容	备注
阶段 0：基础修整	1-2 周	函数、作用域、异常、推导式、OOP；list / dict / set / deque。	独立写字符串统计、容器操作、对象建模。
阶段 1：核心算法	4-6 周	哈希、双指针、二分、窗口、前缀和；每类 12-20 题。	2 分钟内判断候选题型。
阶段 2：结构化搜索	3 周	栈、队列、堆、树、图、DFS / BFS、并查集。	能脱稿写遍历模板与 visited。
阶段 3：组合与最优	2-4 周	递归、回溯、贪心、DP。	能写状态定义、转移、初始值、遍历顺序。
阶段 4：岗位专项	持续	NumPy、PyTorch、项目、医学图像指标。	能完整讲项目、实验、失败与改进。

10 周执行表

周	专题	每日学习	本周产出
1	Python 基础、list / dict / set	5 道基础题 + 30 分钟复盘。	容器速查表、2 个小程序。
2	哈希表	5 题 / 天：先暴力，后 hash 优化。	Two Sum、频率、异位词可脱稿写。
3	双指针	4-5 题 / 天，手动画指针移动。	有序合并、移零、去重、三数之和。
4	二分	4 题 / 天，练普通和边界模板。	左右边界、答案二分。
5	滑动窗口、前缀和	4 题 / 天，记录窗口不变量。	最长 / 最短窗口、和为 K。
6	栈、队列、堆	4 题 / 天，画入栈与出队。	括号、单调栈、层序、Top K。
7	树、图、DFS / BFS	3-4 题 / 天，先背模板再变形。	遍历、岛屿、最短路、课程表。
8	递归、回溯、贪心	3-4 题 / 天，明确选择与撤销。	子集、排列、N 皇后、区间贪心。
9	动态规划	3 题 / 天，先写中文状态。	爬楼梯、打家劫舍、背包雏形。
10	综合模拟	2 套题 + 1 次口述。	错题二刷、项目自述、模拟评分。

每日 / 每周执行方式

- 每天 90-150 分钟：15 分钟旧题、50-80 分钟新题、20 分钟复盘、10 分钟口述。
- 限时标准：简单 20 分钟，中等 35 分钟，困难 50 分钟；超时必须记录卡点。
- 每周末：一次 45-60 分钟模拟笔试；选两题录音讲解，整理三类最常见边界错误。
- 刷题不是记答案：重点问自己“为什么不排序”“为什么这个窗口可收缩”“状态含义是什么”。

4. Python 基础、容器与复杂度清单

函数与面向对象

知识点	必会知识点	常见坑
函数参数	位置参数、默认参数、*args、**kwargs、关键字参数。	不要用 [] / {} 作可变默认参数。
作用域	LEGB; global 与 nonlocal 的区别。	函数内赋值会创建局部变量。
拷贝	copy.copy 浅拷贝; deepcopy 递归复制。	list[:] 对嵌套对象仍是浅拷贝。
迭代器 / 生成器	__iter__、__next__; yield 惰性生成。	生成器通常只能遍历一次。
装饰器	接收函数并返回增强后的函数; wraps 保留元信息。	有参数装饰器多一层函数。
异常 / 上下文	try/except/else/finally; with 自动释放资源。	不要裸 except 吞掉异常。
OOP	封装、继承、多态、property、classmethod、staticmethod。	实例方法第一个参数是 self。

list / dict / set / deque / heapq

容器	用途	常用操作与复杂度	提示
list	顺序存储、下标访问、栈。	append / 尾部 pop 均摊 O(1), 头部 pop O(n)。	不要高频 pop(0)。
dict	映射、频率、索引。	get、items、setdefault; 平均查找 / 插入 O(1)。	键必须可哈希。
set	去重、成员测试、集合运算。	in、add、discard 平均 O(1)。	无序、不能按下标访问。
deque	双端队列、BFS、窗口。	appendleft / popleft / append / pop 为 O(1)。	from collections import deque。
heapq	Top K、优先队列。	heappush / heappop O(log n)。	只保证堆顶最小，不是完整排序。

复杂度直觉

复杂度	来源	10^5 规模的评测
O(1)	固定次数操作、哈希平均查询。	理想。
O(log n)	二分、堆调整。	非常快。
O(n)	一次扫描、双指针、滑窗。	通常安全。
O(n log n)	排序、分治。	通常安全。
O(n^2)	枚举所有配对、冒泡排序。	通常应优化。
O(2^n)/O(n!)	子集、排列、回溯。	只适合 n 很小。

5. 算法思想速通：信号、模板、易错点

5.1 哈希表

识别信号：查找是否出现、计数、补数、分组、去重。

核心做法：维护 seen / count / index；Two Sum 先查 target-x 再写当前值。

易错点：dict.get(key,0)；复杂度是平均 $O(1)$ ，不是任何输入绝对 $O(1)$ 。

5.2 双指针

识别信号：有序、两端夹逼、原地覆盖、回文。

核心做法：左右或快慢指针，每一步至少推进一个指针，保持不变量。

易错点：有序合并不能先拼接再冒泡；循环后补剩余元素。

5.3 二分查找

识别信号：有序或可行性随答案单调变化。

核心做法：统一使用闭区间或半开区间；mid 后正确移动边界。

易错点：找边界时命中 target 仍继续缩小一侧；避免死循环。

5.4 滑动窗口

识别信号：连续子串 / 子数组的最长、最短、K 约束。

核心做法：right 扩张、条件不满足时 left 收缩，维护窗口计数。

易错点：每个指针最多走 n 次，嵌套外观不代表 $O(n^2)$ 。

5.5 前缀和

识别信号：大量区间和、连续和等于目标。

核心做法：pre[i] 表示前 i 项和； $[l,r] = \text{pre}[r+1] - \text{pre}[l]$ 。

易错点：和为 K：先查 prefix-K 次数，再更新当前前缀和。

5.6 排序与贪心

识别信号：排序后局部选择显而易见，如区间结束时间。

核心做法：说明贪心选择为何安全，必要时使用交换论证。

易错点：输入已排序往往提示线性算法，而非一定要重排。

5.7 栈 / 单调栈

识别信号：括号、嵌套、下一个更大 / 更小。

核心做法：普通栈 LIFO；单调栈在弹出时结算答案。

易错点：list.append/pop 是高效栈，避免从开头 pop。

5.8 队列与 BFS

识别信号：按层、无权最短路、扩散。

核心做法：deque 入队出队，入队时写入 visited，按层计距离。

易错点：visited 过晚添加会重复入队。

5.9 DFS、树与图

识别信号：连通块、路径、树遍历。

核心做法：树有空节点基线；图要 visited；网格先判断越界。

易错点：递归深度可能超限，必要时改显式栈。

5.10 递归与回溯

识别信号：所有组合、排列、放置方案。

核心做法：选择 -> 递归 -> 撤销；组合用 start，排列用 used。

易错点：append 后必须 pop；路径不要被错误共享。

5.11 动态规划

识别信号：最优子结构、重叠子问题。

核心做法：状态含义、转移、初始值、遍历顺序四要素。

易错点：先写 dp[i] 的中文含义，不能只背代码。

5.12 并查集

识别信号：动态连通、合并集合、无向环。

核心做法：find 找根，union 合并；路径压缩 + 按秩。

易错点：union 前必须 find；parent 初始指向自己。

5.13 堆

识别信号：持续维护最值、Top K、多路合并。

核心做法：堆顶为当前最小；将新候选 push 回堆。

易错点：值相等且对象不可比时用 (value, serial, object)。

6. AI / 深度学习算法岗位专项

AI 岗不仅问算法，还会追问张量形状、数据流、训练循环、指标、实验可复现性与项目取舍。回答要把理论、代码和实验连接起来。

NumPy 与张量思维

能力	常用操作 / 函数	常见陷阱
形状与广播	shape、reshape、transpose、squeeze / unsqueeze、broadcast。	(B,C,H,W) 的含义；广播如何减少 for 循环。
索引与聚合	切片、布尔索引、where、argmax、sum、mean。	axis 与 keepdims 如何影响形状。
数值稳定	logits、softmax、交叉熵、log-sum-exp。	为何通常直接把 logits 送给交叉熵。
向量化	优先张量操作，少写 Python 逐元素循环。	如何定位性能瓶颈；哪些场景必须循环。

PyTorch 训练流程

模块	核心逻辑	常见错误
数据	Dataset、DataLoader、transform、train/val/test 划分。	数据泄漏；训练验证预处理不一致。
模型	nn.Module、__init__ 定义层、forward 定义计算。	忘记 super().__init__；形状不匹配。
训练	train、zero_grad、backward、optimizer.step。	漏 zero_grad 导致梯度累积。
验证	eval、torch.no_grad、全量汇总指标。	验证仍启用 dropout 或更新 BN。
工程	随机种子、checkpoint、scheduler、混合精度。	不记录配置和数据版本，实验不可复现。

医学图像分割

指标 / 概念	公式或解释	面试常见要点
Dice	$2 P \cap G / (P + G)$ ，越大越好。	类别不平衡时常比 accuracy 有意义。
IoU	$ P \cap G / P \cup G $ ，越大越好。	$Dice = 2IoU / (1 + IoU)$ 。
HD95	表面距离的 95 百分位，越小越好。	衡量边界偏差，削弱离群点影响。
类别不平衡	前景体素远少于背景。	Dice/Focal/加权 CE、采样、patch 裁剪、增强。
2D 与 3D	2D 显存低但上下文少；3D 上下文多但成本高。	说明体素间距、各向异性、patch 与显存权衡。
数据增强	空间、强度、裁剪、噪声等变换。	标签必须同步空间变换；验证集不做随机增强。

项目回答模板

背景与问题 -> 数据规模/划分 -> 预处理与模型选择 -> 损失与指标 -> 对照实验 -> 失败案例和改进 -> 最终结果与局限。不要只报最高分，要解释结果是否稳定、为什么有效。

7. Python 面试重点题库

题库包含选择、简答、编程三类；难度从简单到中等再到困难。选择题与简答题均给出最佳答案及解析。编程题给出可运行 Python 解、复杂度和关键解析。建议先遮住答案。

7.1 选择题：简单

编号	题目	选项	答案	关键解析
S1	a=[1,2]; b=a; b.append(3) 后 a?	A[1,2] B[1,2,3] C报错 DNone	B	同一 list 引用, append 原地修改。
S2	频率统计的正确写法?	A d[x]+=1 B d.get(x,0)+1 C d[x]=d.get(x,0)+1 D d.add(x)	C	get 提供默认值后必须赋回。
S3	平均成员查询 O(1) 的是?	A list B set C index D sorted	B	set 基于哈希, 平均 O(1)。
S4	[x*x for x in range(3)]?	A[0,1,4] B(0,1,4) C[1,4,9] D5	A	依次计算 0、1、4。
S5	高效 FIFO 队列推荐?	A list.pop B list.pop(0) C deque D set	C	deque.popleft 为 O(1)。
S6	函数无 return 时返回?	A0 BFalse C" DNone	D	默认返回 None。

7.2 选择题：中等

编号	题目	选项	答案	关键解析
M1	def f(items=[]): 的风险?	A语法错 B每次新表 C默认表复用 D只能 tuple	C	可变默认参数在调用间共享。
M2	sorted(d) 默认排序什么?	A值 B键 C键值对 D插入顺序	B	字典迭代产生键; 频率排序用 d.items()。
M3	生成器函数的关键标志?	A lambda B yield C递归 D随机访问	B	yield 创建惰性生成器。
M4	with open 的主要价值?	A排序 B自动关闭 C加速 CPU D捕获所有异常	B	上下文退出时释放资源。
M5	is 与 == 的正确说法?	A相同 Bis值 C is身份、==值 D只能数字	C	is 比较对象身份, == 比较相等性。
M6	有序列表查 target 常用?	A冒泡 B二分 C双循环 DDFS	B	有序是二分的直接信号。

7.3 选择题：困难

编号	题目	选项	答案	关键解析
H1	CPU 密集型要利用多核通常优先?	A threading B multiprocessing C asyncio D print	B	CPython GIL 限制纯 Python 线程并行字节码。
H2	asyncio 最适合?	A大量网络/IO等待 B纯 Python 大计算 CGPU训练 D深递归	A	协程在 await 让出执行权, 适合 IO 并发。
H3	MRO 主要决定?	A内存 B方法查找顺序 C哈希 D编码	B	多重继承的 C3 线性化查找顺序。
H4	循环引用的回收依赖?	A禁 gc B循环垃圾回收器 C全局变量 D==	B	分代 GC 处理引用计数无法解决的循环。
H5	heapq 是?	A最大堆 B最小堆 C完整排序 D哈希表	B	堆顶最小, 内部不全排序。
H6	PyTorch 验证推荐?	A train+backward B eval+no_grad C step D随机增强	B	固定推理行为并禁用梯度记录。

7.4 简答题：简单

编号	问题	最佳答案 / 关键知识点
S1	list 与 tuple 的核心区别?	list 可变, 适合增删; tuple 不可变, 适合固定记录。tuple 只有所有元素可哈希时才可作为字典键。
S2	dict.get(key, default) 有何作用?	安全取值, 键不存在时返回 default; 频率统计写 <code>d[x]=d.get(x,0)+1</code> 。
S3	set 为什么可去重?	集合以哈希保存可哈希元素, 相同元素只保留一个; 它不保证顺序。
S4	什么是浅拷贝?	只复制外层容器, 嵌套可变对象仍共享引用; list[:]、dict.copy 均是浅拷贝。
S5	break、continue、pass 区别?	break 退出当前循环; continue 跳过本轮; pass 什么也不做, 仅作语法占位。
S6	为何队列不用 list.pop(0)?	会搬移后续元素, $O(n)$; deque.popleft 是 $O(1)$ 。

7.5 简答题：中等

编号	问题	最佳答案 / 关键知识点
M1	解释装饰器与 functools.wraps。	装饰器接收函数并返回增强后的可调对象, 用于日志、鉴权、缓存。wraps 保留原函数名称、文档等元信息, 便于调试和框架识别。
M2	迭代器与生成器关系?	迭代器实现 <code>__iter__</code> 和 <code>__next__</code> ; 生成器是特殊迭代器, 由 yield 创建, 按需产生值, 通常只能遍历一次。
M3	什么是 GIL?	CPython 中同一时刻通常仅一个线程执行 Python 字节码。线程仍适合 IO; CPU 密集型多用多进程或释放 GIL 的数值库。
M4	何时需要深拷贝?	新对象要独立修改嵌套可变的子对象时需要。若只读或要共享子对象, 浅拷贝更省资源。
M5	为何 hash 查找是平均 $O(1)$?	良好散列让键均匀分布, 单桶期望很短; 碰撞严重可退化, dict 通过扩容保持良好平均性能。
M6	有序合并为何是 $O(m+n)$?	两个指针只前进不回退; 每次输出队首较小者, 它不大于另一表剩余元素, 总推进不超过 $m+n$ 。

7.6 简答题：困难

编号	问题	最佳答案 / 关键知识点
H1	asyncio、线程、进程如何选择?	大量等待 IO 用 asyncio; 简单阻塞 IO 并发可用线程; CPU 密集且需多核用多进程。还要比较共享状态、序列化和第三方库是否释放 GIL。
H2	PyTorch 模型不收敛如何排查?	先单 batch 过拟合; 再查数据/标签、输出与损失、学习率、梯度 0/NaN、train/eval、归一化、类别不平衡、随机种子。一次只改变一个变量。
H3	DP 的四要素?	状态定义、转移方程、初始值/边界、计算顺序。空间压缩前确认新状态依赖哪些旧状态。
H4	DFS 与 BFS 的区别?	DFS 深入路径, 适合连通与枚举; BFS 按层扩展, 在无权图首次到达即为最短步数。带权图需要 Dijkstra 等。
H5	Dice、IoU、HD95 分别衡量?	Dice/IoU 衡量区域重叠, 越大越好; HD95 衡量 95% 表面距离, 越小越好。只看重叠会掩盖边界偏差。
H6	如何让实验可复现?	固定随机种子; 记录代码、配置、数据版本、环境、日志与 checkpoint; 记录 CUDA/cuDNN 确定性设置及代价。

7.7 编程题：简单

E1. 单词出现次数排序 | 简单

题目：忽略大小写，去两端 „!?, 按次数降序、单词升序；不能用 Counter。

最佳答案（可运行 Python 解）：

```
def word_frequency(text):
    counts = {}
    for raw in text.lower().split():
        word = raw.strip(".,!?")
        if word:
            counts[word] = counts.get(word, 0) + 1
    return sorted(counts.items(), key=lambda x: (-x[1], x[0]))

for word, count in word_frequency(
    "Python is great. Python is easy, python is powerful!"):
    print(word, count)
```

复杂度：O(n + k log k) 时间，O(k) 空间。

关键解析：split 保留单词边界；strip 只去两端标点；sorted(counts) 只按键排，必须对 items 指定 key。

E2. 第二大的不同数字 | 简单

题目：不排序返回第二大的不同数字；不存在返回 None。

最佳答案（可运行 Python 解）：

```
def second_largest_distinct(nums):
    largest = second = None
    for number in nums:
        if largest is None or number > largest:
            second, largest = largest, number
        elif number != largest and (second is None or number > second):
            second = number
    return second

print(second_largest_distinct([3, 8, 2, 8, 5, 5])) # 5
```

复杂度：O(n) 时间，O(1) 空间。

关键解析：新最大值出现时旧最大值成为第二大；None 支持全负数；排除与最大值相等的重复数。

E3. 有效括号 | 简单

题目：字符串只含 ()[]{}，判断配对是否有效。

最佳答案（可运行 Python 解）：

```
def is_valid(s):
    match, stack = {}, ['(', ')', '[', ']', '{', '}'], []
    for char in s:
        if char in "([{":
            stack.append(char)
        elif not stack or stack.pop() != match[char]:
            return False
    return not stack

print(is_valid("({})")) # True
```

复杂度：O(n) 时间，O(n) 空间。

关键解析：最近打开的括号必须最先关闭，使用 LIFO 栈；右括号前必须检查空栈。

E4. 两数之和 | 简单

题目：返回和为 target 的两个不同元素下标；假设答案唯一。

最佳答案（可运行 Python 解）：

```
def two_sum(nums, target):
    seen = {}
    for i, number in enumerate(nums):
        if target - number in seen:
            return [seen[target - number], i]
        seen[number] = i
    return []

print(two_sum([2, 7, 11, 15], 9)) # [0, 1]
```

复杂度：平均 $O(n)$ 时间， $O(n)$ 空间。

关键解析：先查补数后写当前数，保证同一元素不被使用两次；暴力双循环为 $O(n^2)$ 。

E5. 合并两个升序列表 | 简单

题目：合并两个升序整数列表，保留重复值；不使用 sort。

最佳答案（可运行 Python 解）：

```
def merge_sorted(a, b):
    result = []
    i = j = 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i]); i += 1
        else:
            result.append(b[j]); j += 1
    return result + a[i:] + b[j:]

print(merge_sorted([1, 3, 5, 7], [2, 3, 6, 8]))
```

复杂度： $O(m+n)$ 时间， $O(m+n)$ 空间。

关键解析：比较两个队首，较小者必可输出；指针只前进，因此无需整体重新排序。

E6. 原地移动零 | 简单

题目：把所有 0 移到末尾，非零元素相对顺序不变。

最佳答案（可运行 Python 解）：

```
def move_zeroes(nums):
    slow = 0
    for fast in range(len(nums)):
        if nums[fast] != 0:
            nums[slow], nums[fast] = nums[fast], nums[slow]
            slow += 1
    return nums

print(move_zeroes([0, 1, 0, 3, 12]))
```

复杂度： $O(n)$ 时间， $O(1)$ 空间。

关键解析：slow 是下一个非零数放置位置；fast 扫描，交换可保持非零元素的相对顺序。

7.8 编程题：中等

M1. 字母异位词分组 | 中等

题目：仅含小写字母；把异位词分组。

最佳答案（可运行 Python 解）：

```
def group_anagrams(words):
    groups = {}
    for word in words:
        count = [0] * 26
        for char in word:
            count[ord(char) - ord('a')] += 1
        groups.setdefault(tuple(count), []).append(word)
    return list(groups.values())

print(group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
```

复杂度： $O(nk)$ 时间， $O(nk)$ 空间。

关键解析：26 位计数元组是签名，避免每个词排序 $O(k \log k)$ ；字符集不限时可用 `sorted(word)`。

M2. 最长无重复子串 | 中等

题目：返回不含重复字符的最长连续子串长度。

最佳答案（可运行 Python 解）：

```
def longest_unique(s):
    last, left, best = {}, 0, 0
    for right, char in enumerate(s):
        if char in last and last[char] >= left:
            left = last[char] + 1
        last[char] = right
        best = max(best, right - left + 1)
    return best

print(longest_unique("abcabcbb")) # 3
```

复杂度： $O(n)$ 时间， $O(\min(n, \text{字符集}))$ 空间。

关键解析：窗口保持无重复；left 跳到重复字符上次出现位置的后一格，且绝不能后退。

M3. 除自身以外数组的乘积 | 中等

题目：`answer[i]` 是除 `nums[i]` 外的所有元素乘积；不能除法。

最佳答案（可运行 Python 解）：

```
def product_except_self(nums):
    answer, prefix = [1] * len(nums), 1
    for i, number in enumerate(nums):
        answer[i], prefix = prefix, prefix * number
    suffix = 1
    for i in range(len(nums) - 1, -1, -1):
        answer[i], suffix = suffix, suffix * nums[i]
    return answer

print(product_except_self([1, 2, 3, 4]))
```

复杂度： $O(n)$ 时间，除输出外 $O(1)$ 空间。

关键解析：第一趟写左侧乘积，第二趟乘右侧乘积；不用除法也自然支持 0。

M4. 查找左右边界 | 中等

题目：有序 nums 中返回 target 首尾下标，不存在返回 [-1,-1]。

最佳答案（可运行 Python 解）：

```
def search_range(nums, target):
    def lower(value):
        left, right = 0, len(nums)
        while left < right:
            mid = (left + right) // 2
            if nums[mid] < value:
                left = mid + 1
            else:
                right = mid
        return left

    left = lower(target)
    right = lower(target + 1) - 1
    return [-1, -1] if left == len(nums) or nums[left] != target else [left, right]

print(search_range([5, 7, 7, 8, 8, 10], 8))
```

复杂度：O(log n) 时间，O(1) 空间。

关键解析：lower 找第一个 $\geq$ value；此实现针对整数用 target+1 求右界，更通用时写 upper_bound。

M5. 二叉树层序遍历 | 中等

题目：返回二叉树每一层的节点值。

最佳答案（可运行 Python 解）：

```
from collections import deque

def level_order(root):
    if not root:
        return []
    result, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result
```

复杂度：O(n) 时间，O(n) 空间。

关键解析：BFS 队列保存候选节点；每层开始先固定 len(queue)，才能准确分层。

M6. 岛屿数量 | 中等

题目：网格由 '1' 与 '0' 构成，统计上下左右连通岛屿。

最佳答案（可运行 Python 解）：

```
def num_islands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])

    def dfs(r, c):
        if not (0 <= r < rows and 0 <= c < cols) or grid[r][c] != '1':
            return
        grid[r][c] = '0'
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            dfs(r + dr, c + dc)

    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)
    return count
```

复杂度：O(RC) 时间，最坏 O(RC) 递归空间。

关键解析：每发现一块陆地就计数并 DFS 淹没整个连通块；不允许改输入时另建 visited。

7.9 编程题：困难

H1. 最小覆盖子串 | 困难

题目：返回 s 中包含 t 所有字符（含次数）的最短子串；没有则空串。

最佳答案（可运行 Python 解）：

```
from collections import Counter

def min_window(s, t):
    if not s or not t:
        return ""
    need, missing = Counter(t), len(t)
    left = start = end = 0
    for right, char in enumerate(s, 1):
        if need[char] > 0:
            missing -= 1
        need[char] -= 1
        while missing == 0:
            if end == 0 or right - left < end - start:
                start, end = left, right
            ch = s[left]
            need[ch] += 1
            if need[ch] > 0:
                missing += 1
            left += 1
        return s[start:end]

print(min_window("ADOBECODEBANC", "ABC")) # BANC
```

复杂度：O(|s|+|t|) 时间，O(字符集) 空间。

关键解析：right 扩张覆盖 t 后尽量收缩 left。need 允许负数，表示窗口中多余字符。

H2. 合并 K 个有序链表 | 困难

题目：合并 k 个升序链表并返回头节点。

最佳答案（可运行 Python 解）：

```
import heapq

class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def merge_k_lists(lists):
    heap, serial = [], 0
    for node in lists:
        if node:
            heapq.heappush(heap, (node.val, serial, node))
            serial += 1
    dummy = tail = ListNode()
    while heap:
        _, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            heapq.heappush(heap, (node.next.val, serial, node.next))
            serial += 1
        tail.next = None
    return dummy.next
```

复杂度：O(N log k) 时间，O(k) 空间。

关键解析：堆中保留每条链表的当前候选；serial 避免相同值时直接比较不可比的节点对象。

H3. 单词拆分 | 困难

题目：判断 `s` 能否由 `word_dict` 单词完全拼接，单词可复用。

最佳答案（可运行 Python 解）：

```
def word_break(s, word_dict):
    words = set(word_dict)
    max_len = max(map(len, words), default=0)
    dp = [False] * (len(s) + 1)
    dp[0] = True
    for end in range(1, len(s) + 1):
        for start in range(max(0, end - max_len), end):
            if dp[start] and s[start:end] in words:
                dp[end] = True
                break
    return dp[-1]

print(word_break("leetcode", ["leet", "code"]))
```

复杂度：O(nL) 时间（L 为最长词，切片成本与实现有关），O(n+字典) 空间。

关键解析：dp[i] 表示前 i 个字符能否拆分；枚举最后一个单词起点，并用 max_len 剪枝。

H4. LRU 缓存 | 困难

题目：固定容量：get 后标记最近使用，put 超容量淘汰最久未使用。

最佳答案（可运行 Python 解）：

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.data = OrderedDict()

    def get(self, key):
        if key not in self.data:
            return -1
        self.data.move_to_end(key)
        return self.data[key]

    def put(self, key, value):
        if key in self.data:
            self.data.move_to_end(key)
        self.data[key] = value
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)
```

复杂度：get / put 平均 O(1)，空间 O(capacity)。

关键解析：OrderedDict 可 O(1) 移到末尾、从头删除。若不允许标准库，使用 hash + 双向链表。

H5. 两个有序数组的中位数 | 困难

题目：求两个升序数组合并后的中位数，要求 O(log(min(m,n)))。

最佳答案（可运行 Python 解）：

```
def median(a, b):
    if len(a) > len(b):
        a, b = b, a
    m, n, lo, hi = len(a), len(b), 0, len(a)
    left_size = (m + n + 1) // 2
    while lo <= hi:
        cut_a = (lo + hi) // 2
        cut_b = left_size - cut_a
        al = float('-inf') if cut_a == 0 else a[cut_a - 1]
        ar = float('inf') if cut_a == m else a[cut_a]
        bl = float('-inf') if cut_b == 0 else b[cut_b - 1]
        br = float('inf') if cut_b == n else b[cut_b]
        if al <= br and bl <= ar:
            return max(al, bl) if (m+n) % 2 else (max(al, bl)+min(ar, br))/2
        if al > br:
            hi = cut_a - 1
        else:
            lo = cut_a + 1

print(median([1, 3], [2]))
```

复杂度： $O(\log(\min(m,n)))$ 时间， $O(1)$ 空间。

关键解析：对短数组二分切分，使左侧数量正确且 $\max(\text{left}) \leq \min(\text{right})$ ，无穷值统一处理边界。

H6. N 皇后 | 困难

题目：返回 $n \times n$ 棋盘放 n 个互不攻击皇后的全部方案。

最佳答案（可运行 Python 解）：

```
def solve_n_queens(n):
    result, cols, d1, d2 = [], set(), set(), set()
    board = [['.' * n for _ in range(n)]]

    def backtrack(row):
        if row == n:
            result.append(''.join(line) for line in board)
            return
        for col in range(n):
            a, b = row - col, row + col
            if col in cols or a in d1 or b in d2:
                continue
            cols.add(col); d1.add(a); d2.add(b); board[row][col] = 'Q'
            backtrack(row + 1)
            board[row][col] = '.'
            cols.remove(col); d1.remove(a); d2.remove(b)

    backtrack(0)
    return result

print(len(solve_n_queens(4))) # 2
```

复杂度：约 $O(n!)$ 时间；递归状态 $O(n)$ ，棋盘 / 输出另计。

关键解析：逐行放一个皇后，用列与两类对角线 $\text{set } O(1)$ 判冲突；回溯必须完整撤销。

8. 复习与模拟面试安排

错题复盘卡

字段	填写要求
题目与题型	写出题目名与第一识别标签：hash、二分、窗口、DFS、DP 等。
第一次思路	记录最初方案和复杂度，不要只写“不会”。
卡点	题型未识别、循环边界、状态定义、Python API、测试遗漏中的哪一类？
正确不变量	例如“窗口内无重复”“堆中每链表仅一个候选”。
边界用例	空、一个、重复、全相同、负数、目标不存在、极端规模。
复习日期	第 1/3/7/14 天重写；第四次仍错就拆成模板练习。

四次模拟面试

时间	内容	要求
第 3 周末	基础选择 + 两道简单算法。	每题先说思路，25 分钟手写，说明复杂度。
第 5 周末	hash、双指针、二分、窗口各一题。	不看笔记，重点检查边界模板。
第 8 周末	树图/回溯一题 + 项目简述。	说清 DFS/BFS 选择及一个失败实验。
第 10 周末	60 分钟综合：选择、简答、两道编码、项目追问。	录音后按正确性、复杂度、表达、测试复盘。

评分量表（每道编码题 10 分）

维度	分值	达标标准
理解与澄清	2	准确指出输入、输出、约束与特殊情况。
算法选择	3	利用条件选择合适策略，并说明替代方案。
实现正确性	3	主逻辑可运行，边界与变量更新一致。
复杂度与表达	2	时间/空间正确，讲解清晰可跟随。

最终检查清单

- 不看资料写出：频率统计、Two Sum、第二大不同数、有序合并、二分边界、最长无重复子串、岛屿数量、爬楼梯 DP。
- 解释：dict/set 平均复杂度、list 与 deque 的队列差异、何时不排序、GIL、Dice 与 HD95。
- 准备 1-2 个项目故事：问题、方法、实验、指标、失败与改进，避免只背模型名称。
- 面试前只复习模板和错题，不冲刺陌生难题；保证睡眠和清晰表达。

结语

面试能力来自可复用的思考路径：看到“有序”想到二分或双指针；看到“连续”想到窗口或前缀和；看到“连通”想到 DFS/BFS/并查集；看到“最优且重复”想到 DP。持续写、说、测、复盘，就会形成稳定的条件反射。